

Namespaces, ConfigMaps i Secrets

Konfiguracja aplikacji i dane wrażliwe w Kubernetes

```
kubectl apply -f namespace.yaml
```



Cel modułu

- pracować we własnym namespace
- oddzielić konfigurację od obrazu kontenera
- utworzyć ConfigMap i Secret
- przekazać dane do aplikacji jako zmienne środowiskowe
- zbudować prostą stronę nginx z wartości z ConfigMap i Secret



Namespace

Namespace to logiczna przestrzeń dla obiektów Kubernetes.

qa-demo

Pody / Service / ConfigMap
Secret

dev

inne obiekty
inny kontekst

prod

inna konfiguracja
inne dostępy

Dla QA namespace ułatwia porządkowanie labów i szybkie filtrowanie obiektów.



Namespace — komendy

```
kubectl create namespace qa-demo  
kubectl get namespaces  
kubectl get pods -n qa-demo
```

CLI → YAML

```
kubectl create namespace qa-demo \  
  --dry-run=client -o yaml > namespace.yaml  
  
kubectl apply -f namespace.yaml
```



Praca w wybranym namespace

```
kubectl get pods -n qa-demo  
kubectl describe pod <pod-name> -n qa-demo
```

Można też ustawić namespace jako domyślny w aktualnym kontekście.

```
kubectl config set-context --current --namespace=qa-demo
```



ConfigMap

ConfigMap przechowuje konfigurację, która nie jest tajna.



Zmiana ConfigMap nie wymaga budowania nowego obrazu kontenera.



ConfigMap — komendy

```
kubectl create configmap page-config -n qa-demo \
  --from-literal=PAGE_TITLE='Kubernetes QA Lab' \
  --from-literal=PAGE_HEADER='Hello from ConfigMap'
```

```
kubectl get configmap -n qa-demo
kubectl describe configmap page-config -n qa-demo
kubectl get configmap page-config -n qa-demo -o yaml
```



Secret

Secret przechowuje dane wrażliwe, np. hasła, tokeny i klucze.



Domyślnie Secret w YAML jest kodowany Base64 — TO NIE JEST SZYFROWANIE!



Przekazanie danych do aplikacji

ConfigMap → env

env:

- name: PAGE_HEADER
valueFrom:
configMapKeyRef:
name: page-config
key: PAGE_HEADER

Secret → env

env:

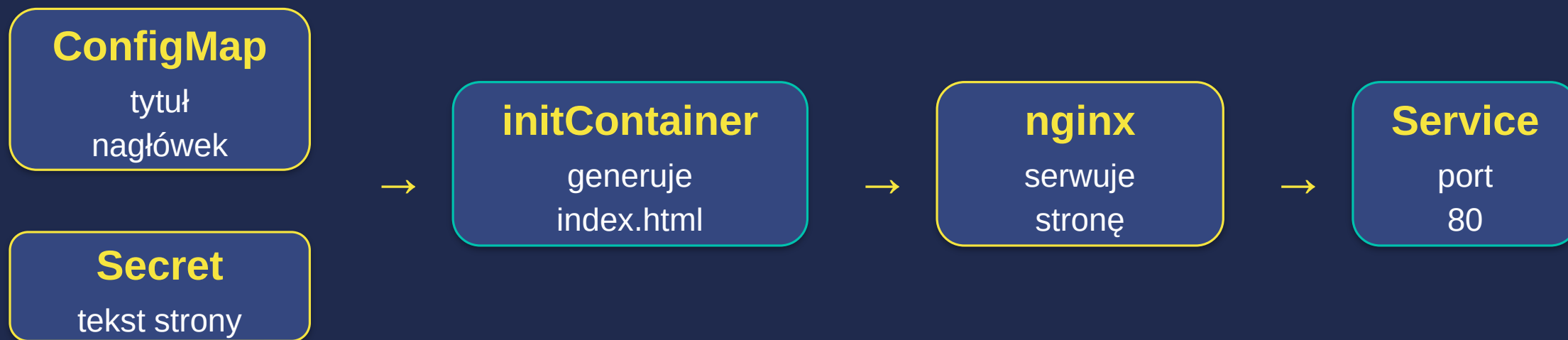
- name: SECRET_MESSAGE
valueFrom:
secretKeyRef:
name: page-secret
key: SECRET_MESSAGE

Aplikacja widzi wartości jako zmienne środowiskowe.



Lab: nginx wyświetla konfigurację i Secret

Efekt końcowy: strona nginx pokazuje tekst z ConfigMap i Secret.



W realnej aplikacji nie wyświetlamy sekretów użytkownikowi — tutaj robimy to wyłącznie demonstracyjnie.



Secret — komendy

```
kubectl create secret generic page-secret -n qa-demo \
  --from-literal=SECRET_MESSAGE='Tekst z Kubernetes Secret'
```

```
kubectl get secrets -n qa-demo
kubectl describe secret page-secret -n qa-demo
kubectl get secret page-secret -n qa-demo -o yaml

echo "$(kubectl get secret page-secret -n qa-demo \
  -o jsonpath='{.data.SECRET_MESSAGE}' | base64 -d)"
```

Do manifestów można używać `stringData`, żeby nie kodować ręcznie Base64.



Struktura katalogu labu

```
05-namespace-configmap-secret/  
├── namespace.yaml  
├── configmap.yaml  
├── secret.yaml  
├── deployment.yaml  
└── service.yaml
```



Manifesty: Namespace i ConfigMap

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: qa-demo
```

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: page-config
  namespace: qa-demo
data:
  PAGE_TITLE: 'Kubernetes QA Lab'
  PAGE_HEADER: 'Hello from ConfigMap'
```



Manifest: Secret

`secret.yaml`

```
apiVersion: v1
kind: Secret
metadata:
  name: page-secret
  namespace: qa-demo
type: Opaque
stringData:
  SECRET_MESSAGE: 'Tekst z Kubernetes Secret'
```

stringData pozwala wpisać wartość czytelnie — Kubernetes zapisze ją jako data/Base64.



Deployment: initContainer generuje index.html

deployment.yaml – fragment

volumes:

- name: html
emptyDir: {}

initContainers:

- name: generate-html
image: busybox:1.36
command: ['sh', '-c']
args:
 - |
cat <<EOF > /html/index.html
<h1>\${PAGE_HEADER}</h1>
<p>\${SECRET_MESSAGE}</p>
EOF



Deployment: wartości z ConfigMap i Secret

initContainer env

env:

- name: PAGE_HEADER
valueFrom:
 configMapKeyRef:
 name: page-config
 key: PAGE_HEADER
- name: SECRET_MESSAGE
valueFrom:
 secretKeyRef:
 name: page-secret
 key: SECRET_MESSAGE

nginx

containers:

- name: nginx
 image: nginx:1.27
 ports:
 - containerPort: 80
 volumeMounts:
 - name: html
 mountPath:
 /usr/share/nginx/html



Manifest: Service

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-secret-demo
  namespace: qa-demo
spec:
  selector:
    app: nginx-secret-demo
  ports:
    - port: 80
      targetPort: 80
```

Service daje stabilny punkt dostępu do Poda nginx.



Uruchomienie labu

```
kubectl apply -f namespace.yaml  
kubectl apply -f configmap.yaml  
kubectl apply -f secret.yaml  
kubectl apply -f deployment.yaml  
kubectl apply -f service.yaml
```

Wszystkie zasoby powstają w namespace qa-demo.



Sprawdzenie działania

```
kubectl get all -n qa-demo  
kubectl get configmap -n qa-demo  
kubectl get secret -n qa-demo  
kubectl describe pod -n qa-demo
```

```
kubectl port-forward -n qa-demo svc/nginx-secret-demo 8080:80  
  
curl http://localhost:8080
```

W przeglądarce: <http://localhost:8080>



Co diagnozujemy w tym labie?

- czy Pod wystartował i initContainer zakończył pracę
- czy ConfigMap i Secret istnieją w tym samym namespace
- czy zmienne środowiskowe zostały poprawnie podpięte
- czy Service wskazuje właściwe Pody
- czy port-forward pozwala wejść na stronę



Do zapamiętania

Namespace

porządkuje
zasoby

ConfigMap

konfiguracja
bez sekretów

Secret

dane
wrażliwe

Dla QA najważniejsze: sprawdzić, czy aplikacja dostała właściwą konfigurację i czy działa w oczekiwanym namespace.

